

Adaptive Cruise Controller for RC Car Using STM32

Erickson A. Mijangos

Department of Electrical Engineering
California State University Fullerton, Fullerton, California
eamijangos@csu.fullerton.edu

Dr. Mike Turi

Department of Electrical Engineering
California State University Fullerton, Fullerton, California
mturi@fullerton.edu

Abstract—This project documents the design and development of an RC Car built from the ground up, as an attempt to implement and test an Adaptive Cruise Control System. Two STM32 Nucleo-Boards from STMicroelectronics are used: the STM32L432KC for the transmitter and the STM32H723ZG for the RC Car. A Python User Interface is developed for controlling the RC Car and performing data acquisition in order to analyze and develop the adaptive cruise controller.

The sensors integrated to the RC Car are the LIS3DH accelerometer and the Garmin LIDAR Lite V3. The accelerometer was used to determine the velocity of the RC Car, and the LIDAR to measure the relative distance to a leading vehicle ahead. During testing, two challenges arose: the velocity calculated by the accelerometer drifted from the true value over time due to sensor noise, and the NRF24L01 lost communication for 1–1.5s every 20ms cycle which affected velocity calculation.

Due to Earth's gravitational influence on the accelerometer, a calibration feature was implemented, but the communication losses made it difficult to confirm its effectiveness. Based on the test results, the adaptive cruise controller could not be fully implemented. Future work includes integrating a Hall Effect sensor and applying a Kalman Filter for sensor fusion to improve velocity accuracy and enable closed-loop speed control.

Index Terms—Adaptive Cruise Control, STM32, PID Controller, LIDAR, NRF24L01, PyQt5, UART.

I. INTRODUCTION

The purpose of this project is to explore electronics, software development, sensor integration, numerical analysis, real-time data acquisition, and primarily control engineering by building an RC Car from the ground up. The RC Car is designed to develop an Adaptive Cruise Controller with data acquisition in order to properly implement it. Figure 1 shows a high-level overview of the entire project — the RC Car listens and sends data back to the transmitter, which is connected to a Python User Interface. The RC Car and transmitter communicate using the NRF24L01 programmable RF module, and the transmitter connects to the computer via USB using the UART protocol to send commands and receive data.

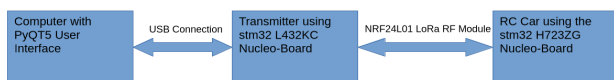


Fig. 1. High-level overview of the project architecture.

The setup of the Transmitter and Python User Interface is shown in Figure 3, and the RC Car with the transmitter is shown in Figure 4. The RC Car integrates the LIS3DH accelerometer to measure velocity and the Garmin LIDAR Lite V3 to measure relative distance.

Based on a simulation study conducted in MATLAB/Simulink at CSULB¹, the Adaptive Cruise Control system is designed to autonomously maintain the desired velocity while detecting and measuring the relative distance to a leading vehicle ahead. As shown in Figure 2, the relative distance and vehicle speed are analyzed to tune the adaptive cruise controller.

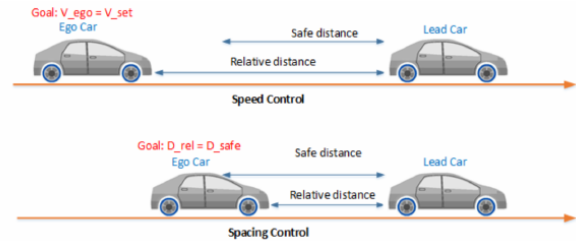


Fig. 2. Speed/Spacing Control to setup Adaptive Cruise Control.

To implement the adaptive cruise controller in hardware, the Garmin LIDAR Lite V3 serves as the primary distance sensor, measuring the relative distance between the Ego Car (the RC Car under ACC control) and the Leading Car (a second RC Car given a specified motion). The LIS3DH accelerometer measures the Ego Car velocity, in which a PID controller can be developed to make the RC Car autonomous. The Leading Car will be given a speed with a small oscillation to simulate realistic driver behavior, as shown in Figure 5.

To achieve the simulated behavior of the adaptive cruise controller, the control system shown in Figure 6 models the Leading Vehicle motion and the Sensor Model, which depends on the relative position and velocity of the leading vehicle and the ego vehicle, so it can be fed into the ego vehicle plant to predetermine next motion. The input signals measured from the relative and ego vehicle parameters are used to determine the ego vehicle Adaptive Cruise Controller are shown in Figure 7.

¹Full report available in the project repository: EE471_ACC_CSULB.pdf.

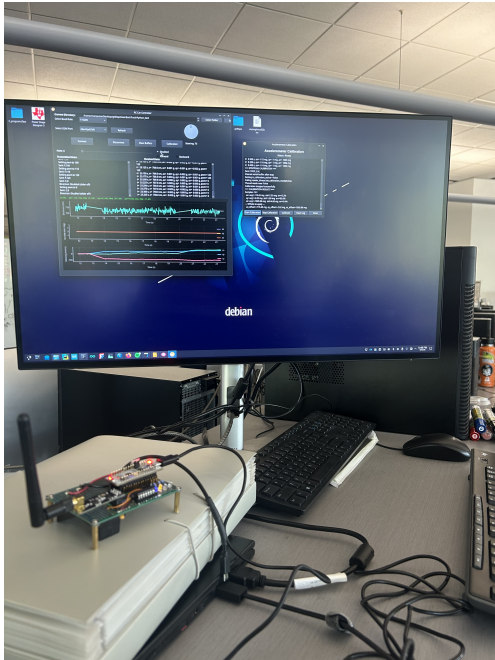


Fig. 3. The Transmitter connected to the Python user interface.

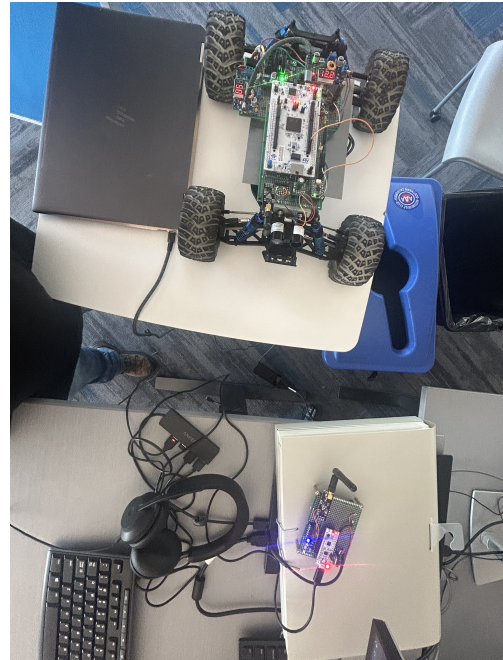


Fig. 4. The Transmitter and RC Car.

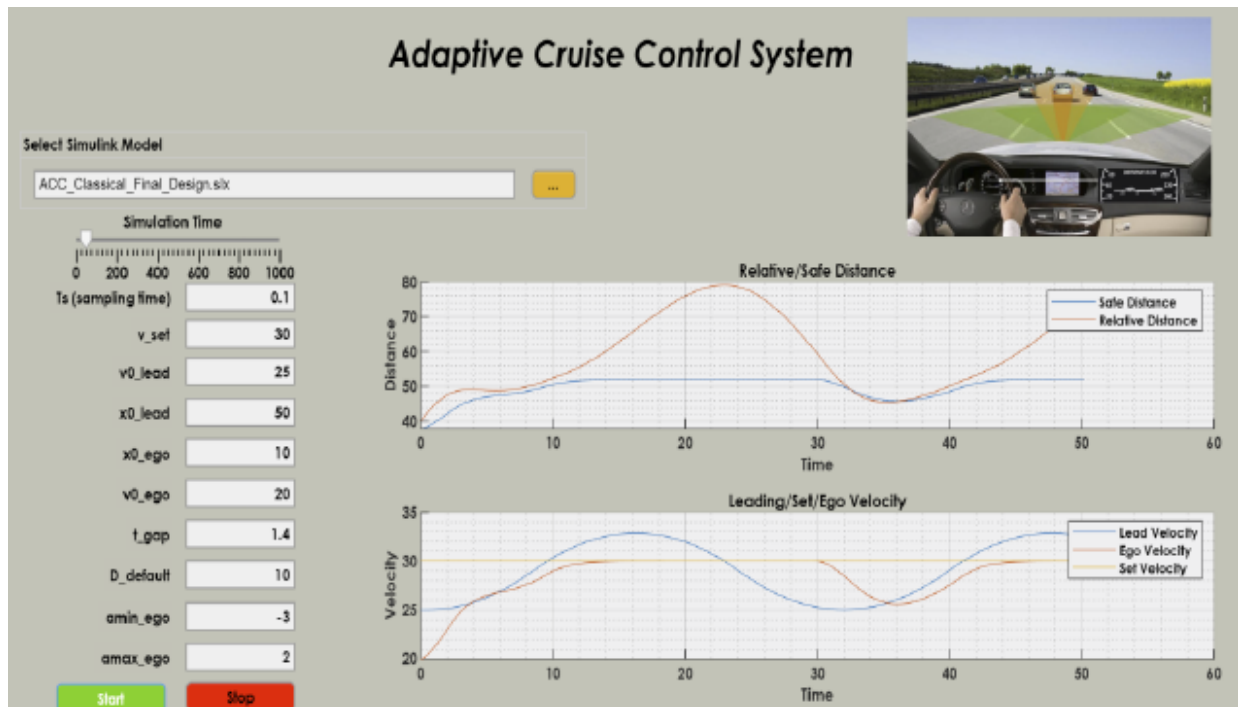


Fig. 5. The User Interface developed in MATLAB and Simulink.

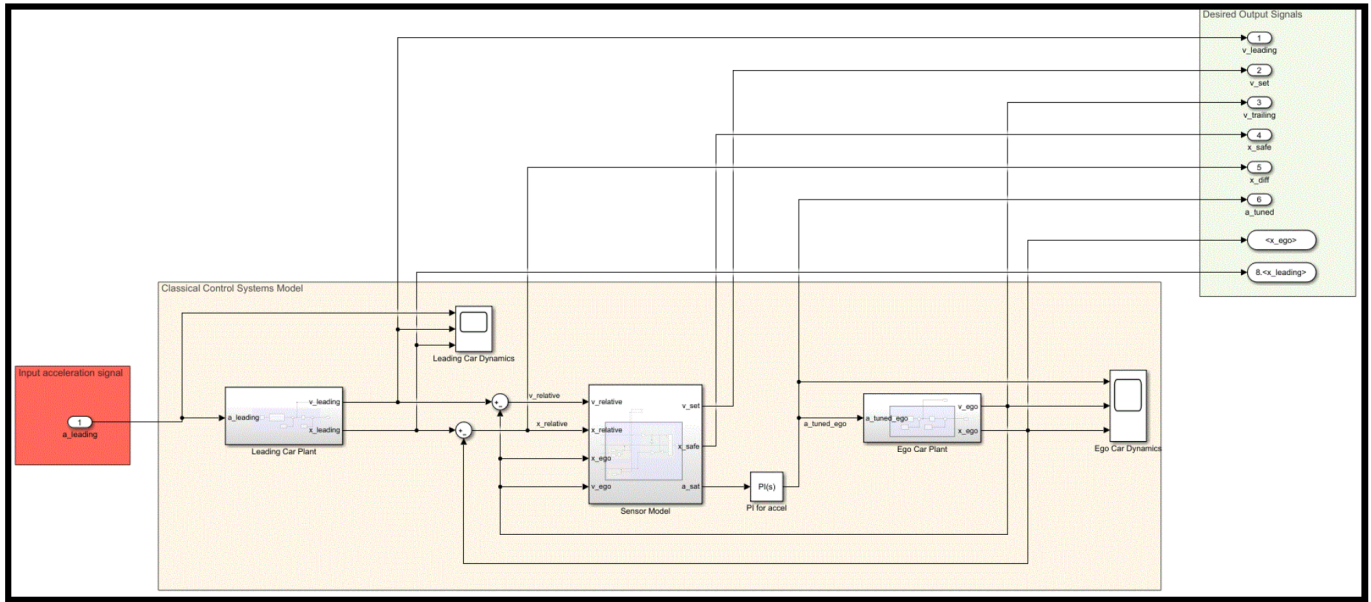


Fig. 6. High-level overview of ACC model.

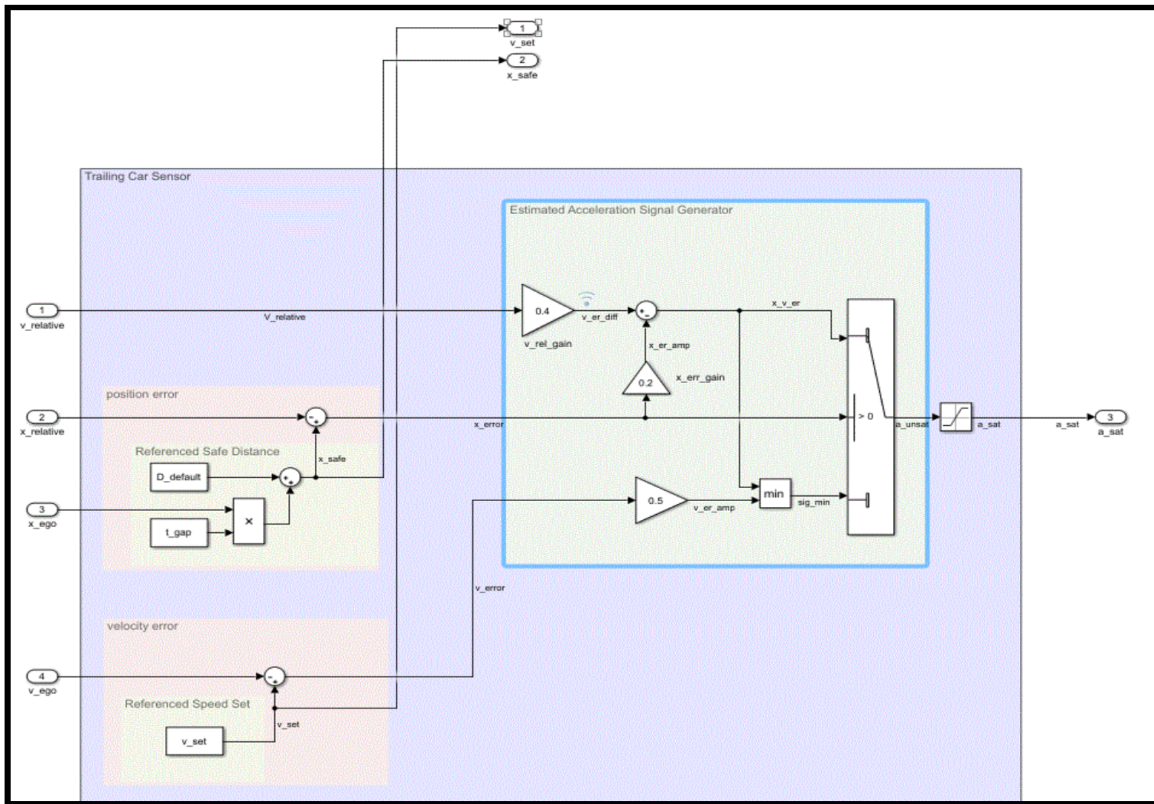


Fig. 7. Block diagram of vehicle plant and derivation for velocity and position motion.

To place the adaptive cruise controller into test, data acquisition with a descent sample rate from both accelerometer and LIDAR are required. To analyze and process the signals collected, the python user interface needs to be implemented in order to acquire data in real time so a proper adaptive cruise controller can be implemented.

II. HARDWARE ARCHITECTURE

A. RC Car Design

The RC Car uses the STM32H723ZG Nucleo-Board, due to having the 550MHz clock speed, to achieve better timer control if necessary, minimize the execution time per instruction/code, accessibilities of more peripherals, and the larger system RAM to store large amount of data for testing purposes. The peripherals used in this MCU are SPI, USART, I2C, timers, and Direct Memory Access to either control sensor and transmit data during development. In Figure 8, shows the STM32H723ZG Nucleo-Board integrated with all the sensors, and power system section.

The Entire RC Car is powered with a 12V battery, but the RC Car is divided by subsections based on the schematic. On the left there a yellow box is drawn to show the MCU along with its sensors, and on the right in the green box shows the power system and the motor driver. The power section is electrically isolated due to the FOD8001 Logic Optocouplers, and the TMH1205S 2W Traco Isolated DC-DC Converter that isolate the power ground and digital ground on the MCU section. This feature is added due to capability to troubleshoot, isolate noise that can influence the sensors during data transmission, and if any power section components/modules are destroyed it would not destroy the MCU and the sensors.

B. Transmitter Design

Subsection content.

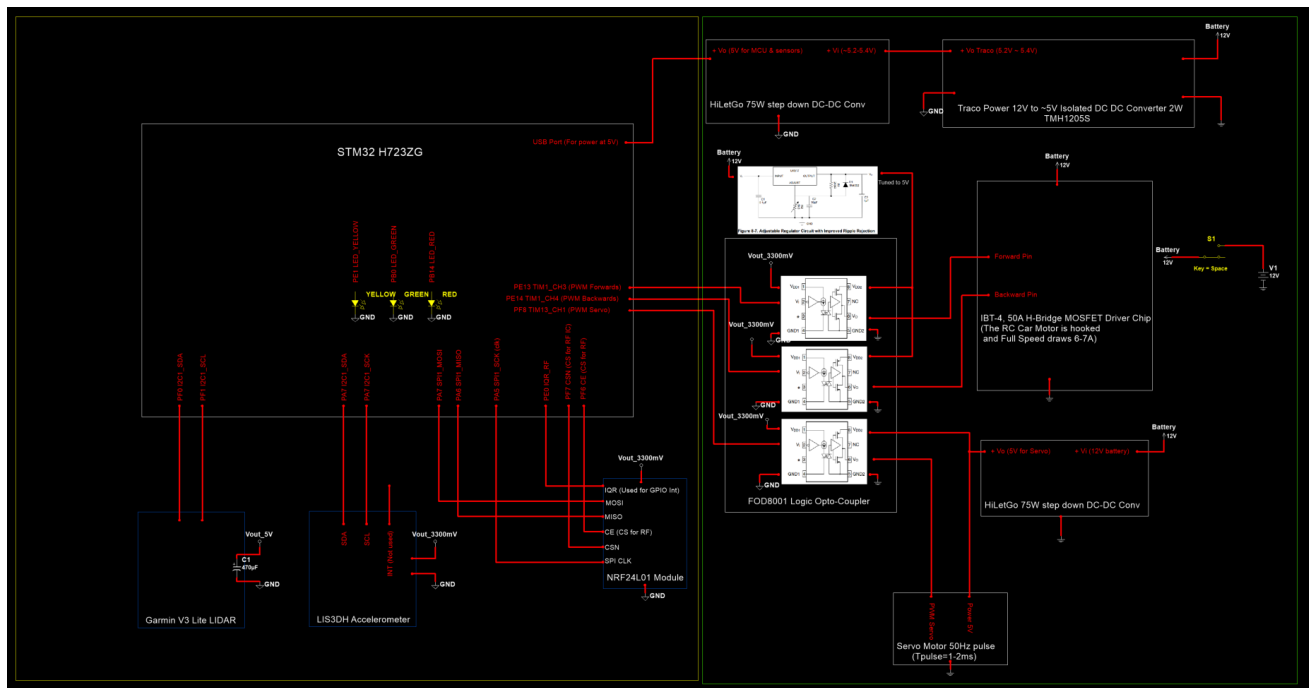


Fig. 8. The Schematic of RC Car.

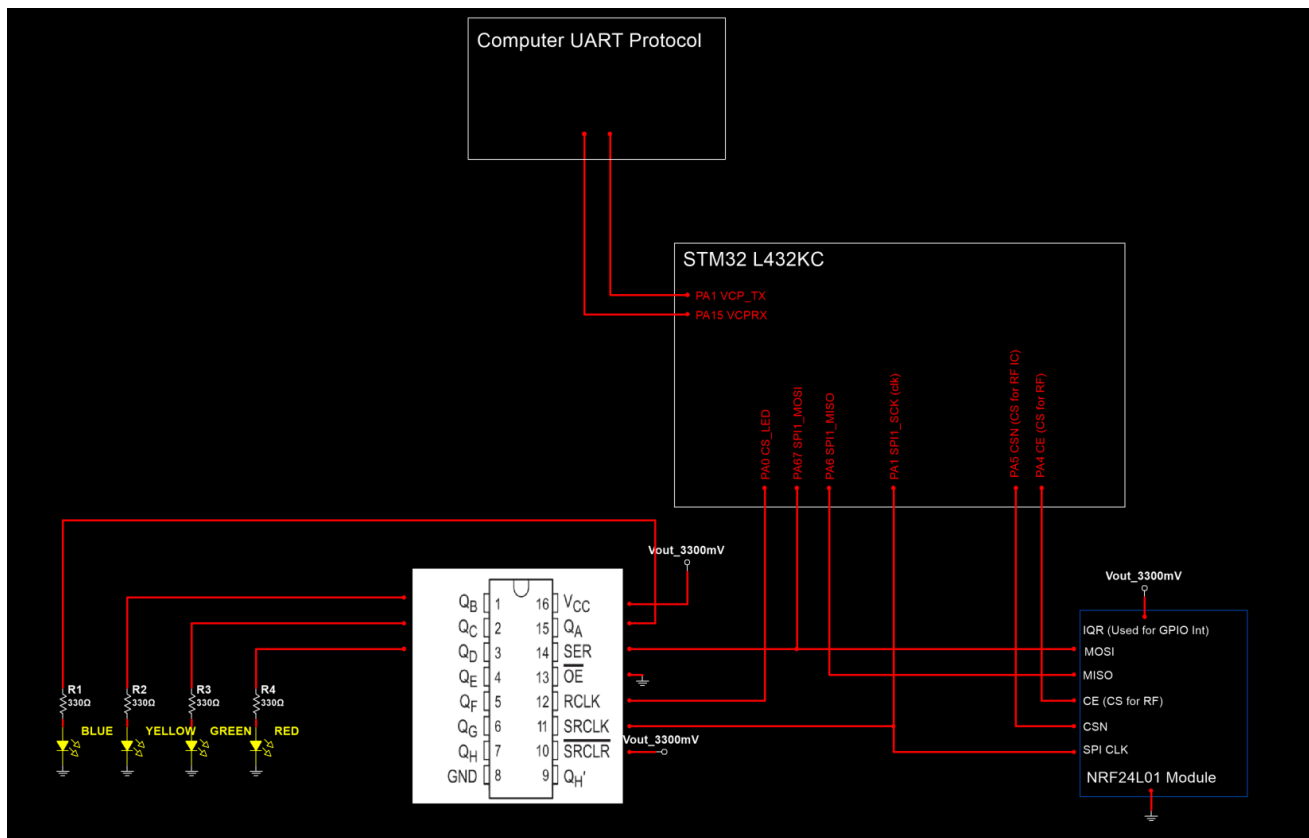


Fig. 9. The Schematic of Transmitter.

III. FIRMWARE DESIGN

A. *RC Car Firmware Design*

Your RC Car firmware section goes here.

B. *Transmitter Firmware Design*

Your Transmitter firmware section goes here.

IV. PYTHON GRAPHICAL USER INTERFACE

Your GUI section goes here.

V. EXPERIMENTAL RESULTS

Your results go here.

VI. CONCLUSION

Your conclusion goes here.

REFERENCES

- [1] STMicroelectronics, *STM32H723ZG Datasheet*, Rev. 4, 2022.
- [2] Garmin Ltd., *LIDAR-Lite v3 Operation Manual*, 2017.